Azure DevOps & Terraform Execution Guide

Step-by-Step Instructions for Local Execution & Azure DevOps CI/CD Pipeline

Prerequisites

Before running any steps, ensure your system has the following tools installed and configured:

• **Terraform CLI (v1.15+)** — Verified installed (v1.15.8 on your system)

• **Azure CLI (v2.x+)** — Required for authenticating and managing Azure resources

• **TFLint & Gitleaks (Optional)** — Recommended for static security and compliance scanning

Method 1: Running Locally (PowerShell / Terminal)

Step 1: Login & Select Azure Subscription

Authenticate with your Azure account and set your active subscription:

```
# 1. Login to Azure
az login

# 2. List subscriptions and set active subscription
az account list --output table
az account set --subscription "<YOUR_SUBSCRIPTION_ID_OR_NAME>"
```

Step 2: Run Code Formatting & Syntax Validation

Validate syntax and structure locally without connecting to remote state backend:

```
# 1. Format all code recursively
terraform fmt -recursive

# 2. Validate Preprod configuration
cd environments/preprod
terraform init -backend=false
terraform validate

# 3. Validate Prod configuration
cd ../prod
terraform init -backend=false
terraform validate
cd ../..
```

Step 3: Deploy Preprod Environment

Navigate to environments/preprod and provision preproduction infrastructure:

> ***NOTE:*** *The remote backend requires Azure Storage Account* ***storagetechaccount*** *in Resource Group* ***rg-ravi****. Ensure these exist before running full init.*

```
cd environments/preprod

# 1. Initialize Terraform with AzureRM Remote State Backend
terraform init

# 2. Generate and review execution plan
terraform plan -out=preprod.tfplan

# 3. Apply infrastructure changes
terraform apply preprod.tfplan
```

Step 4: Deploy Production (Prod) Environment

Navigate to environments/prod and provision production infrastructure:

```
cd environments/prod

# 1. Initialize Terraform
terraform init

# 2. Generate and review execution plan
terraform plan -out=prod.tfplan

# 3. Apply infrastructure changes
terraform apply prod.tfplan
```

Step 5: Destroy / Clean Up Resources (Optional)

To teardown the infrastructure when no longer needed:

```
# Destroy Preprod infrastructure
cd environments/preprod
terraform destroy -auto-approve

# Destroy Prod infrastructure
cd environments/prod
terraform destroy -auto-approve
```

Method 2: Running via Azure DevOps CI/CD Pipeline

Step 1: Create Azure Remote State Storage Account

Execute these Azure CLI commands once to set up backend storage:

```
# Create Resource Group for Terraform State
az group create --name rg-ravi --location eastus

# Create Storage Account
az storage account create --name storagetechaccount --resource-group rg-ravi --location eastus
--sku Standard_LRS

# Create Storage Container
az storage container create --name tfstate --account-name storagetechaccount
```

Step 2: Create Service Connection in Azure DevOps

• Open your Azure DevOps project at **dev.azure.com**

• Navigate to **Project Settings** > **Service Connections**

• Select **New Service Connection** > **Azure Resource Manager** > **Service Principal (automatic)**

• Set Service Connection Name exactly to: azure-service-connection

• Select your Azure Subscription and click **Save**

Step 3: Set Up DevOps Environments & Approvals

• In Azure DevOps, navigate to **Pipelines** > **Environments**

• Create two environments: preprod and prod

• Add **Approvers & Checks** to the prod environment for manual approval before deployment

Step 4: Import Pipeline & Trigger Execution

• Go to **Pipelines** > **New Pipeline**

• Select your repository source (Azure Repos Git / GitHub)

• Select **Existing Azure Pipelines YAML file** and choose /azure-pipelines.yml

• Click **Run** (or push commits / open Pull Requests targeting the main branch)

Pipeline Stage Workflow Overview

Stage	Description	Trigger Condition
1. Validate	Gitleaks secret scan, TFLint analysis, terraform validate	PR / Push to main
2. Preprod Plan	Generates preprod.tfplan execution artifact	After Validate stage
3. Preprod Apply	Applies changes to Preprod environment	Automatic on main branch
4. Prod Plan	Generates prod.tfplan execution artifact	After Preprod Apply stage
5. Prod Apply	Applies changes to Production environment	Requires Manual Approval